

scripts/advance-all-submodules.sh

— User Guide

Last verified: 2026-08-21 (feature 002 build-test-distribute-pipeline, tasks T050–T053 TDD cycle) **Inheritance:** HelixConstitution §11.4.18 (script documentation mandate) + Lava §6.T.3 (no force push) + §6.W (2-mirror scope) + the Decoupled Reusable Architecture rule's submodule-pin policy (as amended by T048 for this pipeline)

Overview

Advances every submodule of a repository to its own upstream's latest commit, one submodule at a time, and writes a **Submodule Advance Record** per submodule describing exactly what happened.

It implements specs/002-build-test-distribute-pipeline/research.md **R-005**'s 7-step ordering verbatim:

1. `git fetch --all` in the submodule
2. compare the currently-pinned commit to the remote default branch's HEAD
3. identical → **no-op** (spec Edge Case: "no newer commit available")
4. different → check the remote HEAD out into the submodule's working tree
5. rebuild + re-run the affected test categories against the advanced state; **on failure the advance is discarded** (the prior pin is checked back out) rather than recording a pin bump that broke the build
6. if the submodule carries local, uncommitted modifications, commit and push them to its own upstream(s)
7. `git add <submodule-path>` in the **parent** repository to record the new pin

The rebuild-and-test-*before*-recording-the-pin ordering is the whole point: it makes the "advance breaks the build" edge case mechanically enforced instead of a documentation-only promise.

Usage

```
# Real use: whole repository, inside a pipeline run
LAVA_PIPELINE_RUN_ID=2026-08-21T14-30-00Z scripts/advance-all-
submodules.sh

# Explicit repository path (also how the hermetic test points it
  at fixtures)
scripts/advance-all-submodules.sh /path/to/repo

# One submodule only, with a custom verification step
LAVA_PIPELINE_RUN_ID=2026-08-21T14-30-00Z \
LAVA_ADVANCE_SUBMODULES="submodules/auth" \
LAVA_ADVANCE_VERIFY_CMD='scripts/ci.sh --changed-only' \
scripts/advance-all-submodules.sh
```

Inputs

Input	Required	Meaning
\$1 (positional)	no	Parent repository path. Default: git rev-parse --show-toplevel. The pipeline run this advance belongs to. Required unless both LAVA_ADVANCE_RECORD_DIR and LAVA_ADVANCE_VERIFY_CMD are supplied — without a run id the script cannot honestly claim to have rebuilt-and-tested anything, since phase-01-build.sh/phase-02-test.sh both require a run id with an existing report.json.
LAVA_PIPELINE_RUN_ID	conditionally	Where Submodule Advance Records are written. Default: .lava-ci-evidence/pipeline-runs/<run_id>/submodule-advances (per data-model.md).
LAVA_ADVANCE_RECORD_DIR	no	The R-005 step-5 rebuild-and-test command, run once per advanced submodule via
LAVA_ADVANCE_VERIFY_CMD	no	

Input	Required	Meaning
LAVA_ADVANCE_SUBMODULES	no	bash -c, with \$1 = the submodule's absolute path and \$2 = the parent repository root. Zero exit ⇒ the advanced state still builds and passes. Default: phase-01-build.sh <run_id> <repo> && phase-02-test.sh <run_id> <repo>. Whitespace-separated allow-list of submodule paths (as they appear in git submodule status). Default: every submodule.

Nothing is hardcoded (§6.R): the repository path, submodule set, remote name, and default branch name are all resolved at runtime from git itself — the default branch in particular comes from the remote's own HEAD symref (git ls-remote --symref), because this project's submodules do not all share one default branch name.

Outputs

One JSON file per submodule at <record-dir>/<sanitized-submodule-path>.json, conforming to specs/002-build-test-distribute-pipeline/contracts/submodule-advance-record.schema.json:

```
{
  "submodule_name": "submodules/auth",
  "old_commit": "32a80e0a...",
  "new_commit": "9f1c2b47...",
  "local_modifications_pushed": false,
  "parent_pin_updated": true,
  "outcome": "ADVANCED"
}
```

outcome is one of:

Outcome	Meaning	parent_pin_updated
NO_NEWER_COMMIT	Pin already equals the upstream default branch HEAD; nothing was done (step 3).	false

Outcome	Meaning	parent_pin_updated
ADVANCED	old_commit == new_commit. Fetched, checked out, verified, (optionally) pushed local modifications, and staged the new pin.	true
REJECTED_BREAKING_CHANGE	The advanced state failed the step-5 rebuild-and-test (or could not be checked out). Advance discarded; prior pin restored.	false
REJECTED_PUSH_CONFLICT	The submodule's own local modifications could not be pushed as a fast-forward. Refused rather than overwritten (FR-016). Advance discarded; prior pin restored.	false

The record's own writer enforces the schema's all0f/if-then invariant — a REJECTED_* outcome can never be emitted alongside parent_pin_updated: true.

Aggregating these records into the run's report.json (submodule_advances[]) is phase-07-closure.sh's job (T055), not this script's.

Exit codes

Code	Meaning
0	Every submodule ended ADVANCED or NO_NEWER_COMMIT.
1	At least one submodule was rejected or could not be processed.
2	

Code	Meaning
	Usage/configuration error (bad repo path, missing run id with no overrides, unwritable record directory). Nothing was attempted.

Side effects

- **In each submodule:** `git fetch`, a detached-HEAD checkout, and — only when local modifications exist — a commit plus a fast-forward push to every configured remote (§6.W: GitHub + GitLab).
- **In the parent repository:** `git add <submodule-path>` only. It **never commits and never pushes the parent** — that is `phase-07-closure.sh`'s job, deliberately separated so a review gate sits between "pins staged" and "pins pushed".

Safety properties

- **Never force-pushes, never rewrites history, never bypasses hooks or signing** (§6.T.3). A push that cannot fast-forward is a `REJECTED_PUSH_CONFLICT` — the refusal is the correct outcome.
- **A rejected advance is discarded entirely, never partially applied.** Both rejection paths check the submodule back out to its prior pinned commit, so the parent repository is left exactly as it was found.
- **Never destroys local work.** When local modifications had to be committed before a push that then failed, the resulting commit is preserved under `refs/lava-advance-rescue/<id>` inside the submodule *before* HEAD is moved back — reachable, not reflog-only. Restoration uses a plain checkout, never a discarding one.
- **No privilege escalation** — plain git as the calling user (§6.U).

Hermetic test

`tests/pipeline/test_advance_all_submodules.sh` exercises all four R-005 branches against **disposable git fixtures** built under `mktemp -d` — a throwaway parent repo, a throwaway submodule, and a local bare repo standing in for that submodule's upstream. It never touches this repository's real submodules/ tree and never reaches any network host.

Case	Fixture setup	Asserted
1	Pin already at upstream HEAD	<code>NO_NEWER_COMMIT</code> , <code>old_commit == new_commit</code> , pin untouched, exit 0

Case	Fixture setup	Asserted
2	Upstream one commit ahead, verify passes	ADVANCED, submodule HEAD and parent index gitlink both moved to upstream HEAD, exit 0
3	Upstream one commit ahead, verify fails	REJECTED_BREAKING_CHANGE, submodule HEAD restored, parent pin unchanged, exit 1
4	Local modification present; the verify hook lands a concurrent commit on the fixture upstream, making the subsequent push a genuine non-fast-forward	REJECTED_PUSH_CONFLICT, pin unchanged, local_modifications_pushed: false, rescue ref present and still containing the local edit, fixture upstream tip un-overwritten, exit 1

Every assertion is on **real git state** (`git ls-files -s`, `git rev-parse HEAD`, `git show <ref>:<file>`), not merely on what the JSON record claims — so a record that lies about what it did is caught by the test rather than believed.

Falsifiability rehearsal (2026-08-21, T052 GREEN step): the step-5 failure branch was mutated to stage the parent pin (`git add`) instead of calling `restore_pin`, while still writing `parent_pin_updated: false` into the record — i.e. a record that lies. Case 3 failed with:

```
FAIL: case3(breaking-change): submodule HEAD restored to the prior
pin expected '2244ef88...', got '20b78dcc...'
FAIL: case3(breaking-change): parent pin NOT updated expected
'2244ef88...', got '20b78dcc...'
```

Cases 1, 2 and 4 stayed green, so the failure was localized to exactly the mutated path. The mutation was reverted and the suite returned to 36/36 PASS.

2026-08-22 pre-T054 audit: hardening + open review items

This script is gated behind Human Checkpoint #2 (task T054) because of its blast radius. Ahead of that review it was audited against disposable git fixtures (bare local repos standing in for upstreams; no network, no real submodule touched). Twelve failure modes were demonstrated and fixed; every one has a case in `tests/pipeline/test_advance_all_submodules_hardening.sh`.

What was fixed

#	Failure mode	What it did
1	Uninitialized submodule → the script operated on the PARENT repo	<code>git -C <empty-dir> rev-parse --git-dir</code> succeeds by walking UP, so the guard never fired. The parent repository was detached from its branch and its tree replaced with its own origin/HEAD; the run reported <code>ADVANCED / parent_pin_updated: true / exit 0</code> . Now guarded by <code>is_own_repo_root</code> , which requires the directory to be the root of its own work tree. Its <code>stderr</code> was discarded and zero parsed rows produced no submodules found – nothing to do + exit 0 . Its exit status is now checked before its output is parsed; a failure is exit 2 with git's own diagnostic.
2	A failing git submodule status read as "nothing to do"	<code>grep -qw</code> treats the path as a word-boundary regex, so <code>LAVA_ADVANCE_SUBMODULES=submodules/auth-extra</code> also selected <code>submodules/auth</code> . Now an exact whole-token comparison.
3	The allow-list widened runs instead of narrowing them	"Remote HEAD differs from the pin" is not "remote HEAD is newer". This project pins submodules to side branches on purpose (root CLAUDE.md records Containers pinned on <code>lava-pin/2026-05-07-pkg-vm</code>), and moving such a pin to origin/HEAD drops the pinned commit. Now refused via <code>merge-base --is-ancestor</code> , before anything is checked out.
4	A pin move that is not a fast-forward was recorded as ADVANCED	<code>pushed</code> was set from having <i>entered</i> step 6. The step-4 checkout can legitimately leave <code>git add -A</code> with nothing to stage (e.g. the new upstream commit adds a <code>.gitignore</code> rule covering the operator's untracked file). Now derived from a commit actually being created.
5	local_modifications_pushed: true when nothing was committed or pushed	
6		On a multi-mirror push where mirror 1 accepted and mirror 2

#	Failure mode	What it did
	local_modifications_pushed: false when one mirror already had the work	refused, the record stated the opposite of the upstream's observable state — and force-push is forbidden, so it cannot be taken back. Now reports true and prints an explicit MIRROR DIVERGENCE warning.
7	Pushed one remote's default branch name at every remote	The branch was resolved once, from the preferred remote. A mirror whose default is main got a stray master branch while its real default received nothing. Now resolved per remote.
8	(cannot fast-forward) asserted for every push failure	Branch protection, a server-side hook, an auth failure and an unreachable host all reported as a non-fast-forward while git's real message was discarded. git's own message is now quoted verbatim.
9	Parent-staging failure after a successful push lost the commit locally	HEAD was moved back off a commit that was already published, with no rescue ref — reflog-only. A rescue ref is now created on that path too.
10	**A governance-deny record-write failure was swallowed by `	
11	A KILLED verify command was indistinguishable from a breaking change	An OOM kill or timeout produced a byte-identical record to a genuine test failure. The exit status (and the signal, for ≥ 128) is now stated in the run log.
12	A failed restore-to-prior-pin was a lone stderr line	The submodule is then left on the <i>rejected</i> commit and the parent tree is dirty; in a 25-submodule run that line scrolls away. Now counted and named in the run summary (N not restored to the prior pin).

Open items for the T054 reviewer — NOT fixed here

1. **The record schema's outcome enum cannot express several real refusals.** submodule-advance-record.schema.json offers five values. A pin move that is not a fast-forward, an uninitialized submodule, an unreachable upstream, a remote whose default branch cannot be resolved, and a parent index that cannot be

staged match none of them. Rather than assert a cause that did not occur, those refusals are reported on stderr, counted, and reflected in the exit code, but leave **no Submodule Advance Record** — the same "invisible at rest" problem the REFUSED_GOVERNANCE_DENY value was added to solve. Closing it needs new enum values (e.g. REFUSED_NOT_FAST_FORWARD, NOT_EVALUATED), which is a specs/ change and therefore a reviewer decision. The step-7 staging-failure path still records REJECTED_BREAKING_CHANGE, which is the closest available value and not the real cause; the run log says so explicitly.

2. **The governance deny-list holds only constitution, but root CLAUDE.md declares far more than one submodule off-limits to an unattended run.** The Decoupled Reusable Architecture rule states that submodule pins are *"explicit and frozen by default"*, that *"updating the pin is a deliberate PR"*, and — verbatim — that *"submodule fetch/pull is an EXPLICIT operator action, never automatic. No git hooks that silently update pins, no git submodule update --remote in any release script."* The §6.AC HelixQA waiver names exactly one exception (submodules/helixqa, always-track-upstream per operator decision Q9) and states that the other sixteen *"remain pins-frozen by default; their bumps still require deliberate per-submodule operator authorization."* This repository currently has **25** submodules. As written, a default run advances 24 of them. The reviewer must decide whether the deny-list should instead be an **allow-list** — the default-deny posture the constitution describes — with submodules/helixqa as its only standing member.
3. **Step 6 commits with git add -A and pushes to a real upstream.** `has_local_mods` comes from `git status --porcelain`, which reports untracked files. Any untracked, non-ignored file in a submodule — a build output, a scratch note, a file a `.gitignore` does not yet cover — is therefore committed and published to that submodule's own upstream under the message chore: local modifications carried forward by advance-all-submodules. This is R-005 step 6 as specified, but its interaction with §6.H (credential inviolability) deserves an explicit reviewer decision: an allow-list of paths, `git add -u` (tracked files only), or an operator confirmation gate.
4. **Never yet run against a real submodule upstream.** Every finding above came from disposable local fixtures. The fixtures cannot reproduce real-upstream behaviour such as authentication prompts, server-side hooks on GitHub/GitLab, or `ls-remote` against a host that redirects.

Maintenance

When this script is modified, update this document in the same commit (CM-SCRIPT-DOCS-SYNC requires it). Per §11.4.18 the documentation MUST stay in sync with the codebase.

Cross-references

- `scripts/advance-all-submodules.sh` — the script itself
- `tests/pipeline/test_advance_all_submodules.sh` — its hermetic test suite (R-005 branches + governance deny)
- `tests/pipeline/test_advance_all_submodules_hardening.sh` — the 2026-08-22 audit's failure-mode suite
- `specs/002-build-test-distribute-pipeline/research.md` — R-005 (the 7-step ordering this implements)
- `specs/002-build-test-distribute-pipeline/data-model.md` — the Submodule Advance Record entity
- `specs/002-build-test-distribute-pipeline/contracts/submodule-advance-record.schema.json` — the record schema
- `scripts/pipeline/lib/evidence.sh` — supplies the record-filename sanitizer this script reuses
- `scripts/pipeline/phase-07-closure.sh` — the caller (T055); owns parent-repo commit/push and `report.json` aggregation
- `Lava CLAUDE.md` §6.T.3 (no force push), §6.W (2-mirror scope), §6.U (no privilege escalation)